Embedded Data Structure
(Lecture Note 14)

Sorting Algorithms (1)

2025. 4. 29

Prof. Wonjun Kim

School of Electrical and Electronics Engineering

Before Starting …

- **Now, we will focus on handling data**
  - **Three main topics : sorting, search, and graph model**

[Given data]

Sorting :

1→ 2 → 3 → 4 → 5 → …

Search :

9→ 2 → 3 → 7 → 4 → …

Graph :

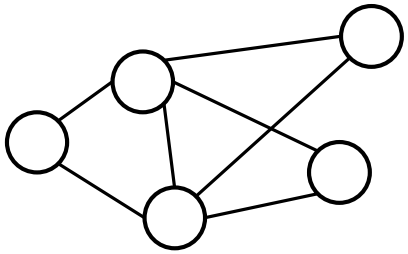
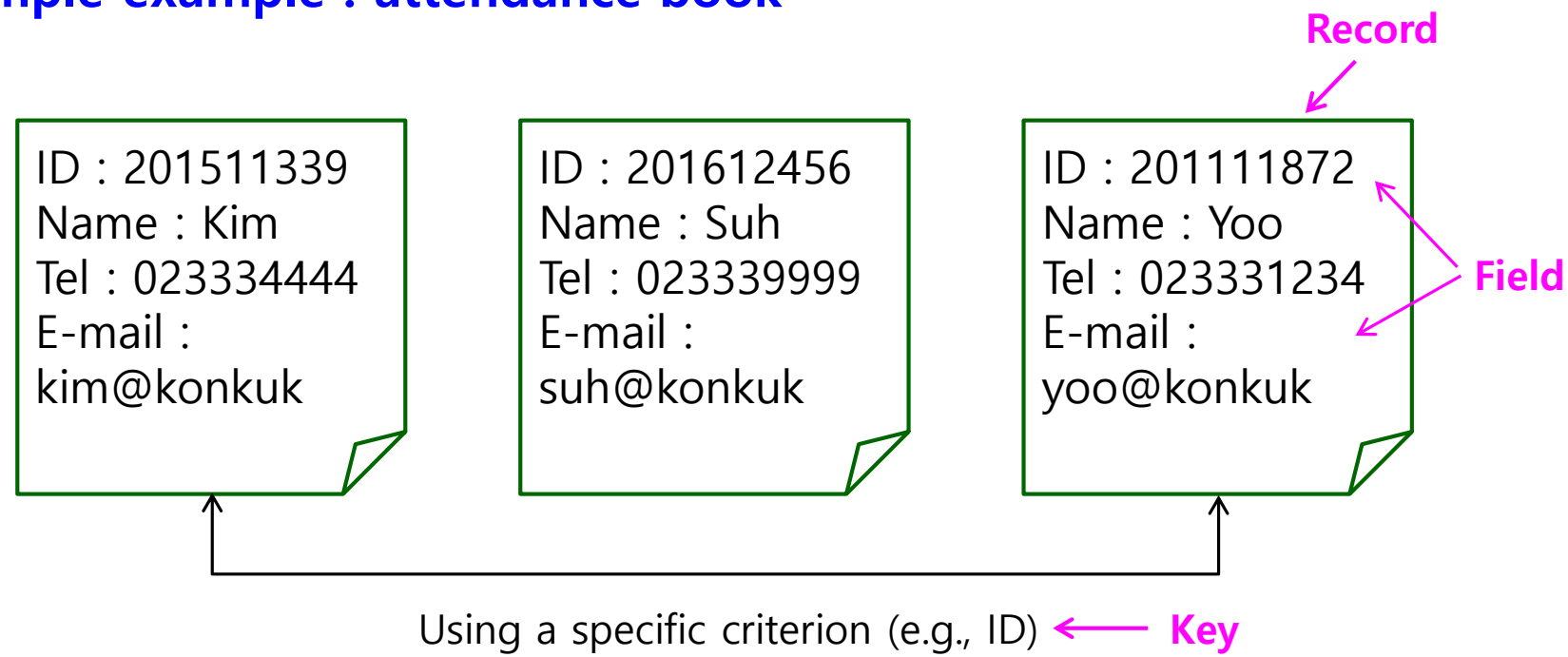


Image from : http://www.the-pr.co.kr/news/articleView.html?idxno=12893

Introduction (1/3)

- **Sorting : re-arrangement of the given data**
  - **Simple example : attendance book**



* There are two operations : **decision** and **exchange**

➔ All the sorting algorithms are based on the combination of them !

Introduction (2/3)

▪ Definition by using C programming

- **Make your own structure, e.g., "student" for attendance book**

Define your structure :

```
typedef struct _student
{
    int id;
    int tel_number;

    char name[20];
    char e_mail[100];
}student;
```

Record

*** Stability of sorting algorithms**

: For example, key → age

If age is the same,

then sorting by the ID number

(Algorithm is stable !)

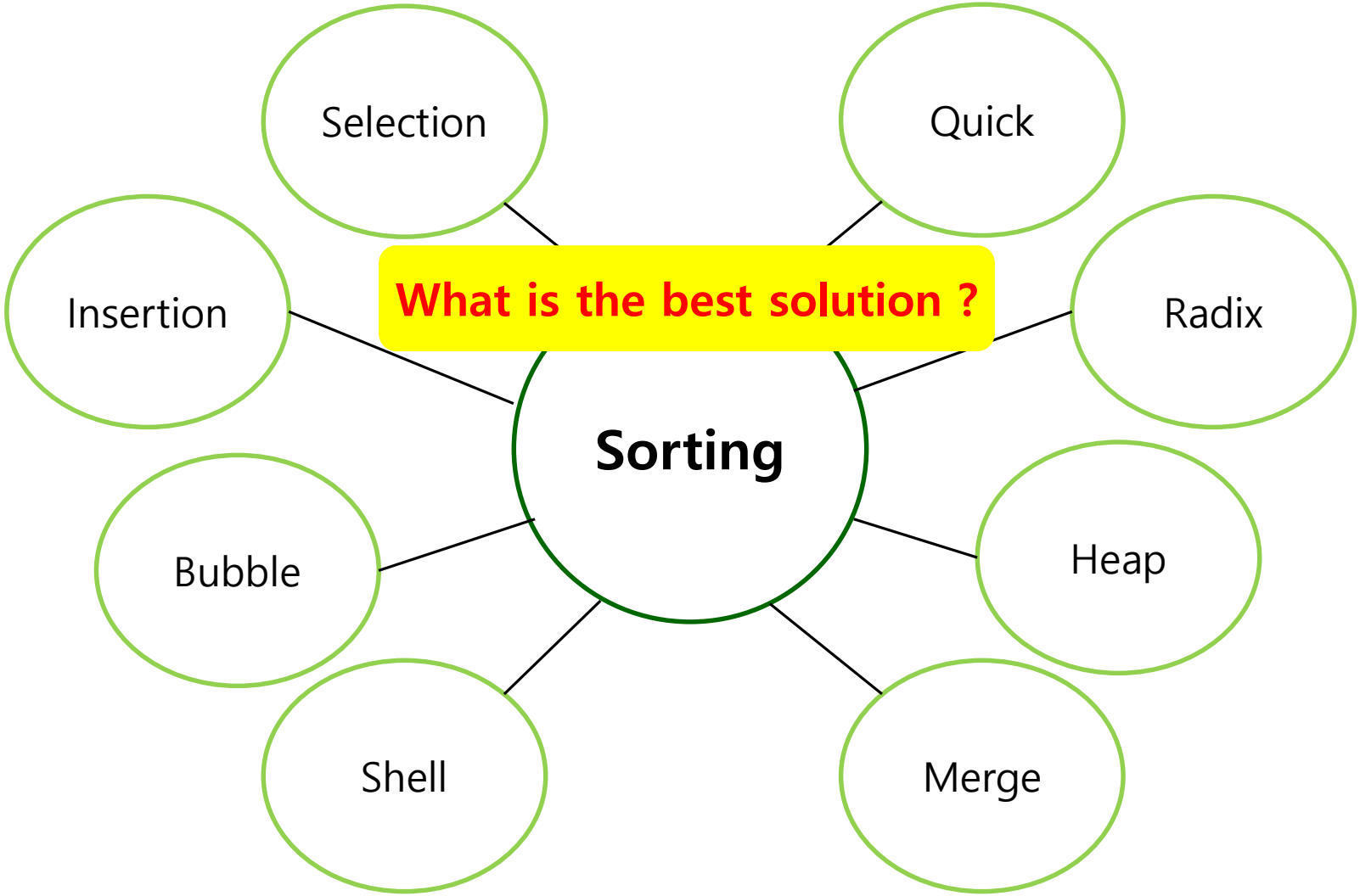
➔ Multi-key based sorting

※ Sorting order

➔ Ascending / descending order

Introduction (3/3)

- **Diversity of sorting algorithms**



Selection Sort (1/8)

- **Basic sorting algorithm**
  - **Find the minimum (N) → find the minimum (N-1) → …**

Pseudo code :

1) j = 0;
2) if j == N-2, then stop
3) Find the minimum from j to N-1 (store it to the min)
4) Exchange j and min
5) j++ and go to the step 2)

※ For example, given "945139"

9^1 4 5 1 3 9^2

↓

1 4 5 9^1 3 9^2

↓

1 3 5 9^1 4 9^2

1 3 4 9^1 5 9^2

↓

1 3 4 5 9^1 9^2

Finish sorting

Is this sorting stable ?
(Think about more)

Selection Sort (2/8)

▪ C Implementation

- **Using the previous pseudo code and implement it**

```
void select_sort(int *a, int N)
{
    int min;
    int min_idx;
    int x, y;

    for(y = 0; y<N-1; y++){
        min_idx = y;
        min = a[y];
        for(x = y+1; x<N; x++){
            if(min > a[x]){
                min = a[x];
                min_idx = x;
            }
        }
    }
}
```

Index moving

Range for minimum

Two statements are required

Fill the blanks !

Selection Sort (3/8)

KU KONKUK UNIVERSITY

▪ Practice : selection sort

- **Using the randomly generated number**

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE  10

void main()
{
   int number[SIZE];

   Make your code !

}
```

Evaluate the processing time according to different sizes (QueryPerformance (see LecNote 1))

Selection Sort (4/8)

KU KONKUK UNIVERSITY

- **Characteristics of selection sort**
  - **It requires a lot comparisons (how many ?)**
  - **It requires a relatively small # of exchanges (how many ?)**

➔ **How about big-O notation ?**

※ Analysis of the processing time (using the tick)

N	Sorted	Random	Half-sorted	Reverse
10	0	0	0	0
100	1	1	1	1
500	15	16	16	18
1000	61	62	62	69
2000	245	246	246	276

What is the worst case ? Why ?

Selection Sort (5/8)

- **Generalization of select_sort function**

  - **Using ① void pointer and ② function pointer**

Sort only for "int"	Sort for all the types
Store key : int min	void *min; min = malloc(width);
Array elem : a[y]	*(type)((char *)base + y*width)
Comparison : min > a[x]	fcmp(min, (char *)base + x*width) > 0
Assign : a[y] = min	memcpy((char *)base + y*width, min, width)

※ Is it possible to conduct pointer operations for "void" type ?

```
int intcmp(const void *a, const void *b)
{
    return (*(int *)a - *(int *)b);
}
```

What does it mean ?

Change the type what you want !

Selection Sort (6/8)

▪ Implementation : gen_select_sort

```
void gen_select_sort(void *base, size_t nelem, size_t width,
                     int (*fcmp)(const void *, const void *))
{
    void *min;
    int min_idx;
    int i, j;

    min = malloc(width);
    for(y = 0; y<nelem-1; y++)
    {
        min_idx = y;
        memcpy(min, (char *)base+y*width, width);
        for(x = y+1; x<nelem; x++0
        {
            if(fcmp(min, (char *)base + x*width) > 0)
            {
                memcpy(min, (char *)base + x*width, width);
                min_idx = x;
            }
        }
```

→ **What does it mean ?**

→ **What does it mean ?**

→ **What does it mean ?**

Selection Sort (7/8)

- **Implementation : gen_select_sort – cont'd**

⋮

```
        memcpy((char *)base + min_idx*width, (char *)base + y*width, width);
        memcpy((char *)base + y*width, min, width);
    }
    free(min);
}
```

What does it mean ?

You can use it as gen_select_sort(a, N, [ ], [ ]);

- It slows compared to the select_sort() → why ?

: function call for comparison, copy for = operation , etc.

- It performs without modifying contents of the function

Selection Sort (8/8)

- **Practice : gen_select_sort()**
  - **Using the randomly generated number**

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE  10

void main()
{
   int number[SIZE];

   ----------------------------------------------------------------
   |                                                              |
   |                      Make your code !                        |
   |                                                              |
   ----------------------------------------------------------------

}
```

Compare the processing time with a function "select_sort()" (QueryPerformance (see LecNote 1))

Bubble Sort (1/3)

- **Basic concept of bubble sort**
  - **Compare two elements and find the maximum (or minimum)**

Pseudo code :

```
1)  i = 0;
2)  if i == N-1, then stop
3)  j = 1;
4)  If j == N-i, then go to the step 7)
5)  If a[j-1] > a[j], then exchange two values
6)  j++ and go to the step 4)
7)  i++ and go to the step 2)
```

If i = 0, 9 4 5 1 → 4 9 5 1 → 4 5 9 1 → 4 5 1 9

If i = 1, 4 5 1 9 → … Try it your self !

Bubble Sort (2/3)

▪ C Implementation

- **Using the previous pseudo code as follows:**

```
void bubble_sort(int *a, int N)
{
    int i, j, t;

    for(i = 0; i<N-1; i++)
    {
        for(j = 1; j<N-i; j++)
        {
            if(a[j-1] > a[j])
            {
                t = a[j-1];
                [                ]
                [                ]
            }
        }
    }
}
```

If your want to sort in a descending manner, then [ ]

Bubble Sort (3/3)

- **Characteristics of bubble sort**
  - **It requires a lot "compare" and "exchange"**
  - **For "sorted" case, check the result compared with selection sort**

➔ **How about big-O notation ?**

※ Analysis of the processing time (using the tick)

N	Sorted	Random	Half-sorted	Reverse
10	0	0	0	0
100	1	1	1	1
500	18	27	21	37
1000	73	109	83	145
2000	294	437	336	578

Variations of the processing time is relatively small

Summary

- **Sorting algorithms : data management**
  - **There is no optimal solution for all the cases**
  - **Remember the generalization scheme using void pointers**